

Bitcoin: Computer Science Functioning

NECKER

November 10, 2025

«It is not real, yet it works: a complexity invisible to many that makes it alive and untamable by anyone.»

– NECKER

Contents

1	What Bitcoin Consists of In Concrete Terms	2
2	Block-Chain	3
2.1	Concatenation	3
3	Miners	4
3.1	Basic Data Structure	4
3.2	Transaction Tree Construction	5
3.3	Finding the PoW	5
3.4	Publication	6
3.5	Fork Problem	6
3.6	Network Reliability and Consensus	7
3.6.1	Rules	7
3.6.2	Incentives	7
3.6.3	Disincentives	7
4	Hardware and Efficiency	7
4.1	Mining Pools and Variance	7
5	Bitcoin Transaction	9
5.1	Legacy Serialization	9
5.2	SegWit (BIP-141)	10
6	Resources:	11

1 What Bitcoin Consists of In Concrete Terms

Bitcoin, from a computer science perspective, is not a file or a cloneable string of bits, but an immutable, **distributed ledger**. Its essence is based on the following concepts:

- **Immaterial and Decentralized Nature:** Being a virtual currency, it has no material counterpart (like gold or state reserves). Its value is guaranteed by **supply/demand**, **algorithmic scarcity** (supply limited to 21 million units), and the **cryptographic security** of its protocol.
- **It is Not a Cloneable String of Bits (The Double Spending Problem):**
 - **The Problem:** If Bitcoin were simply a string of bits (like a digital file), it could be copied and spent multiple times (**Double Spending Problem**).
 - **The Solution:** Bitcoin solves this problem by not existing as a physical entity, but as a **Transaction History**. A Bitcoin is, in essence, the cryptographically and mathematically verified proof of an unspent transaction output (**UTXO**) that has not yet been spent, recorded permanently on the Blockchain.
- **Concrete Definition: The Blockchain:**
 - Bitcoin consists of a **Blockchain**: a chain of **Blocks** containing groups of validated **Transactions**.
 - Ownership of a Bitcoin is not given by possessing a digital object, but by possessing a **Private Key** that allows cryptographically signing (authorizing) the spending of a specific UTXO recorded on the chain.
- **Network Infrastructure**
 - **Peer-to-peer protocol:** Bitcoin is a P2P protocol that allows independent nodes to exchange blocks and transactions without central intermediaries. Nodes communicate via gossip and autonomously validate received information.
 - **Node types:**
 - * *full* nodes that maintain the complete chain (Ledger)
 - * *pruned* nodes that maintain only part of the chain to save space
 - * *light*/SPV client nodes that only verify Merkle proofs for transactions
 - * The full size of the ledger is approximately 500 GB, while a pruned node drastically reduces the required storage.
 - **Mempool and propagation:**
 - * Once a transaction is made, it enters the mempool of the nodes waiting to be included in a block; indeed, a transaction is not immediate and must first be confirmed, meaning included in a block (we will see later that this will not suffice → see **Fork Problem**).
 - * When a transaction is in the mempool, it means it has been broadcast to all nodes in the network. Sorting is often based on fees, which influence the probability and timing of confirmation (the fee is optional; it works because users generally want the transaction to be approved as soon as possible).
 - **Role of miners/validators:** Miners can be anyone and compete to solve the PoW (discussed below → see PoW) and propose blocks; they are not central "authorities" but incentivized participants (reward + fee). Security stems from broad consensus and the difficulty of the PoW.

- **Wallets and keys:** Wallets manage pairs of keys (public/private). Keeping the private key safe is equivalent to controlling the associated UTXOs: losing the key means losing access to the funds.
- **Trust and consensus:** Trust is distributed: unless 50%+1 of the computing power is controlled by a malicious actor, the network remains secure. Consensus rules are encoded in open-source software (e.g., Bitcoin Core) and are enforced autonomously by the nodes (see **Network Reliability and Consensus**).
- **Auxiliary infrastructure:** Besides nodes, there are services like block explorers, wallet providers, exchanges, and secondary layers (e.g., Lightning) that improve usability but can introduce optional centralized points.

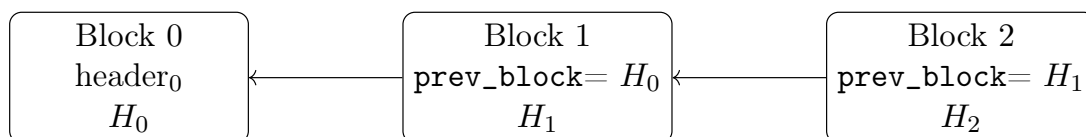
2 Block-Chain

The **blockchain** is a data structure that allows the secure, transparent, and immutable recording of all transactions made on the Bitcoin network. It is composed of a sequence of **blocks**, each containing a set of validated transactions, a reference to the previous block (via its hash), and a **Proof of Work (PoW)** that guarantees its integrity and security. The chain starts from the **genesis block** created by Satoshi Nakamoto on January 3, 2009. To verify if a transaction is valid, it must be:

- **Included In A Valid Block:** it must be included in a block that respects all the network consensus rules.
- **Reachable from Genesis Block:** the block in which the transaction is inserted must be reachable from the main chain, and from it, one can trace back to the first block (genesis block).

2.1 Concatenation

The **concatenation** of blocks occurs by including the hash of the previous block (hash of the block header **N-1**) in the current block header. This creates a chain in which every block depends on the previous one, ensuring the integrity of the chain: modifying a block would require recalculating the PoW of all subsequent blocks, making it practically impossible to alter the transaction history without the consensus of the majority of the network.



Verification: to confirm that the link is valid, the hash of the previous header is recalculated $H_{i-1} = \text{SHA256}(\text{SHA256}(\text{header}_{i-1}))$ and compared with the value stored in **prev_block**. If they match, the block correctly points to the previous one.

3 Miners

"Miners" are network participants who dedicate computing power to the production of new blocks. Their main job is to collect valid transactions from the mempool and build a candidate block (block template).

3.1 Basic Data Structure

Construction of the block template: the miner selects transactions from the mempool (at their discretion), calculates the Merkle root, and creates the coinbase transaction containing the block reward (subsidy) and the collected fees.

Block structure:

Field	Bytes	Description
Magic	4	0xD9B4BEF9 (mainnet)
Size	4	Block size
Header	80	See below
Tx Count	1-9 (varint)	Number of transactions
Tx	variable	Raw transactions

- **Magic:** network identifier (mainnet, testnet, regtest), signals that it is the Bitcoin protocol.
- **Size:** total size of the block in bytes.
- **Header:** 80 fixed bytes (see the table below).
- **Tx Count:** number of included transactions (varint).
- **Tx:** list of actual transactions (in raw format).

Header structure:

Offset	Bytes	Field	Type	Notes
0	4	Version	int32_t	bits 0-3: version
4	32	prev_block	char[32]	Previous block hash (LE)
36	32	merkle_root	char[32]	Merkle root (LE)
68	4	timestamp	uint32_t	Unix timestamp
72	4	bits	uint32_t	Compacted target
76	4	nonce	uint32_t	PoW counter

- **Version:** indicates the version of the Bitcoin software used to create the block.
- **prev_block:** hash of the previous block (little-endian).
- **merkle_root:** root of the tree of included transactions.
- **timestamp:** creation time of the block (the only variable value alongside the nonce; it can be adjusted slightly to try to find the correct nonce).
- **bits:** represents the number of zeroes in the difficulty target in compressed format (first byte = exponent + last three bytes = mantissa). This value is the same for all miners working on the same block; every 2016 blocks (about every 2 weeks), it is recalculated based on the speed at which previous blocks were found to keep the average block generation time constant (about 10 min).

Retarget formula:

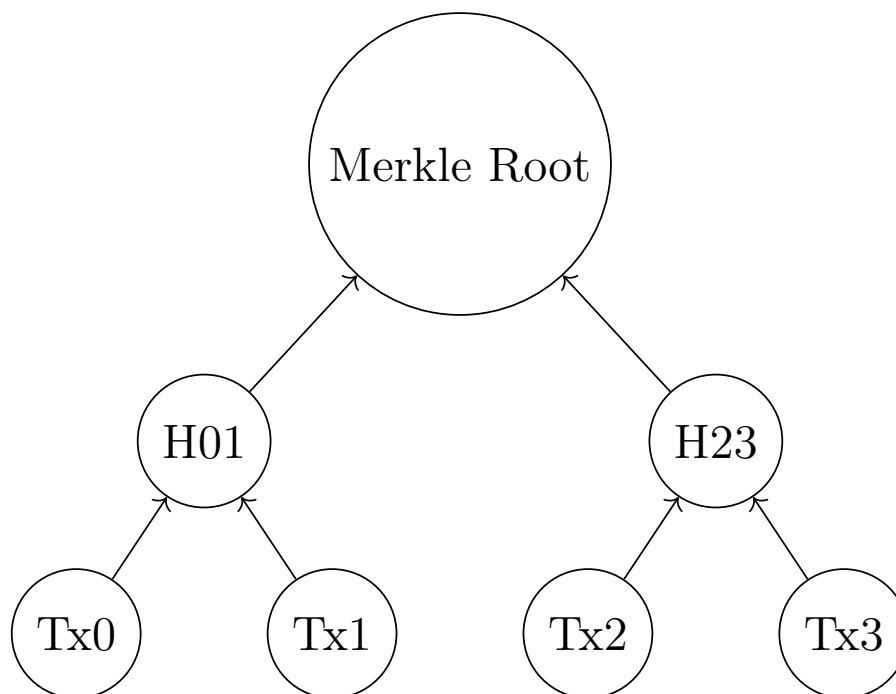
```
new = old * (actual_time / 1209600)
clamped between 0.25x and 4x
```

- **nonce:** variable counter used for the PoW (the number found that, when inserted into the header, verifies the completion of the puzzle).

3.2 Transaction Tree Construction

Note: when constructing the block, we must ensure that the included transactions cannot be modified (or removed) once the block is published. To do this, a data structure called a **Merkle Tree** is used:

- **Merkle Tree:** a binary tree structure where each leaf represents the hash of a transaction, and each internal node is the concatenated hash of its children. The root of the tree (Merkle root) is included in the block header to guarantee the integrity of all data. If even a single transaction is modified, the root hash will change, invalidating the block.



3.3 Finding the PoW

Note: once the block structure is created, we must discourage malicious users from creating fake blocks (see **Network Reliability and Consensus**). For this reason, the miner must find a **nonce** value such that the hash of the block header is lower than the difficulty target specified in the **bits** field. This process is known as **Proof of Work (PoW)**:

- **Double SHA-256:** PoW requires calculating the double-SHA256 on the block header. The miner varies the **nonce** field (and, if necessary, the **timestamp** by a few seconds/minutes) and repeats the calculation until finding a hash that satisfies the target (finding a hash with at least a certain number of leading zeroes dictated by the **bits** field).

$$H := \text{SHA256}(\text{SHA256}(\text{header}))$$

where **header** is the concatenation of the block header fields (:: denotes the concatenation symbol):

$$header = \text{Version} :: \text{prev_block} :: \text{merkle_root} :: \text{timestamp} :: \text{bits} :: \text{nonce}$$

Interpreting the hash as a non-negative 256-bit integer H_{int} , the validity condition for the PoW is:

$$H_{\text{int}} \leq \text{target}$$

The compact **bits** field encodes the target according to the formula (E = exponent, C = 3-byte coefficient):

$$\text{target} = C \times 2^{8(E-3)}$$

Difficulty is defined as:

$$\text{difficulty} = \frac{\text{target}_{\text{max}}}{\text{target}}, \quad \text{target}_{\text{max}} = 0xFFFF \times 2^{8(0x1d-3)}$$

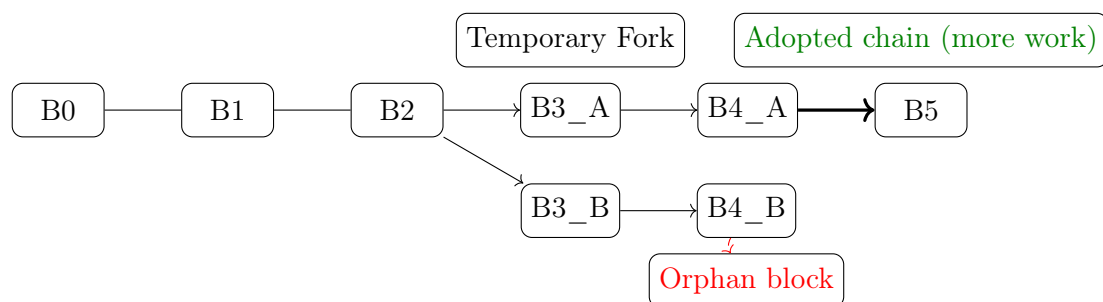
3.4 Publication

Note: when a block is published, the ones who decide whether it is valid or not are the network nodes that receive it (meaning other miners). If the block satisfies all the consensus rules (correct format, valid PoW, valid transactions, no double-spend, etc.), it is added to the node's local chain. The rule determining which block gets added is: whoever finds the value first wins, unless multiple miners find the number at the same time (see **Fork Problem**).

- **Verification:** nodes verify the PoW, the validity of the transactions, and the integrity of the block.
- **Addition to the chain:** if valid, the block is added to the node's local blockchain. For a block to be finalized, at least 51% of the network's total computing power must accept it.
- **Propagation:** the block is broadcast to other nodes in the network.

3.5 Fork Problem

Note: when a block is published, it does not reach everyone instantaneously; if two miners find a valid block almost simultaneously within a short timeframe (e.g., a miner in China and one in Mexico), a **temporary fork** of the chain is created. Nodes receiving the blocks at different times might have diverging opinions on which is the latest valid block. Resolution occurs when a miner finds the next block, extending one of the two chains; the network adopts the longest chain (with the most cumulative work) as valid, and the blocks from the other chain become orphans.



- **confirmation waiting period:** to mitigate the risk of forks, users wait for multiple confirmations (subsequent blocks) before considering a transaction final (typically 6 confirmations).
- **reward maturation period:** for the same reason, the block reward (subsidy + fees) is considered final only after 100 blocks (about 16 hours), to prevent a miner from spending a reward from a block that later becomes an orphan.

3.6 Network Reliability and Consensus

Note: since there is no central certifier guaranteeing network integrity, Bitcoin relies on a distributed consensus mechanism based on strict rules and economic incentives to ensure all nodes agree on the blockchain state. Ideally, the only way for an attacker to compromise the network is to control more than 50% +1 of the total computing power (Geopolitically, the problem already exists since more than 50% of the computing power is allocated in China).

3.6.1 Rules

- **No Double-Spending:** to ensure a transaction is not spent twice, each transaction must reference unspent outputs (UTXOs).
- **Validity of transactions:** each transaction must be signed with the private key corresponding to the input UTXO, and must comply with scripting rules (ScriptPubKey and ScriptSig).
- **Discarding Transactions:** upon validating a block, when computing the next block to mine, nodes discard all transactions already included in previous blocks.
- **finalization:** if multiple competing chains exist, nodes adopt the chain with the most cumulative work (longest chain).

3.6.2 Incentives

To incentivize miners to participate honestly in the consensus process, Bitcoin uses a reward system:

- **Block Reward:** each mined valid block grants the miner a fixed reward (subsidy) that halves every 210,000 blocks (about every 4 years) in an event called the "halving" (we are currently at approximately 3.125 BTC per mined block).
- **Transaction Fees:** besides the block reward, miners collect the fees associated with the transactions included in the block. Fees are paid by users to incentivize miners to include their transactions more quickly.

3.6.3 Disincentives

- **Operational Costs:** mining requires specialized hardware and significant energy consumption. These operational costs act as a deterrent against economic attacks, since an attacker would have to incur high costs to control a significant portion of the network's computing power.

4 Hardware and Efficiency

- **Evolution:** CPU → GPU → FPGA → specialized ASICs for SHA-256. Today, mining is dominated by ASICs designed to maximize throughput and energy efficiency.
- **Metrics:** energy consumption (Joules) per unit of work (TH/s) and cost per hash influence profitability.

4.1 Mining Pools and Variance

- **Why pools exist:** mining has high variance (finding a block is rare but highly rewarding), so miners join *pools* to stabilize revenues. The pool aggregates computing power and distributes earnings among participants according to preset rules.

- **Remuneration schemes:** payments to participants can follow different schemes, the most common being:
 - **PPS (Pay Per Share):** the miner receives a fixed payment for each valid *share* sent to the pool, regardless of whether the pool actually finds a block. This minimizes variance but transfers the risk to the pool operator.
 - **PPLNS (Pay Per Last N Shares):** payment is proportional to the last N shares sent; this scheme rewards miners who remain loyal to the pool and reduces the possibility of "pool hopping".
- **Share:** a *share* is a proof of work at a reduced difficulty (target enforced by the pool) proving a miner contributed useful work. Shares are not valid blocks for the network but serve the pool to account for each miner's contribution.
- **Role of the pool operator:** the pool coordinates mining jobs (assigns header/nonce ranges), collects shares, sends the resulting block to the network, and manages reward distribution. This introduces a coordination point (potential centralization) and requires trust in correct payment by the operator.
- **Payment even without a block found:** some schemes (e.g., PPS) provide regular payments even if the pool does not find blocks; practically, the pool absorbs the statistical risk and distributes it among participants.
- **Stratum Protocol:** the standard protocol used between miners and pools to distribute jobs, submit shares, and reduce communication overhead. Stratum is lightweight, efficient, and allows frequently updated jobs.

5 Bitcoin Transaction

5.1 Legacy Serialization

Bitcoin transactions are serialized in a specific binary format that allows efficient transmission across the network. The structure of a Bitcoin transaction includes several key fields, including version, inputs, outputs, and locktime. Below is the detailed structure of a Bitcoin transaction in legacy raw format (pre-SegWit but still widely used):

Furthermore, let us remember that the currency is effectively the transaction itself; therefore, a user owns bitcoin if they own the transactions that allocate it to them (UTXO).

Table 1: Raw transaction format (legacy)

Field	Bytes	Description
Version	4	Version (little-endian)
Input Count	varint	Number of inputs
Input[i]:	variable	Input entries
- Prev Hash	32	Hash of the previous tx
- Prev Index	4	Index of the spent output
- ScriptSig Len	varint	Length of the ScriptSig
- ScriptSig	varint	ScriptSig
- Sequence	4	Sequence
Output Count	varint	Number of outputs
Output[j]:	variable	Output entries
- Value	8	Value in satoshis
- ScriptPK Len	varint	Length of the scriptPubKey
- ScriptPK	varint	ScriptPubKey
LockTime	4	Lock time of the block/transaction

- **Version:** indicates the transaction version.
- **Input Count:** number of inputs in the transaction expressed as a varint (e.g., if I need to spend **5 BTC** and I have 3 transactions worth: **1 BTC**, **1.5 BTC**, **3 BTC**, to perform the transaction I must include all 3).
- **Input[i]:**
 - **Prev Hash:** hash of the previous transaction from which the output is being spent.
 - **Prev Index:** index of the specific output in the previous transaction.
 - **ScriptSig Len:** length of the ScriptSig.
 - **ScriptSig:** script providing proof of ownership of the output (digital signature).
 - **Sequence:** field used for advanced features like RBF (Replace-By-Fee).
- **Output Count:** number of outputs in the transaction expressed as a varint (e.g., if I want to send **4 BTC** to a recipient and receive **1 BTC** as change, I will have 2 outputs; if I include the fee, I will also have the fee output).
- **Output[j]:**
 - **Value:** value of the output in satoshis (1 BTC = 100,000,000 satoshis).
 - **ScriptPubKey Len:** length of the ScriptPubKey.
 - **ScriptPubKey:** script specifying the conditions to spend the output (e.g., recipient's address).
- **LockTime:** specifies the minimum time or block before the transaction can be included in a block.

5.2 SegWit (BIP-141)

Segregated Witness (SegWit) is a Bitcoin protocol upgrade introduced with BIP-141 that modifies the transaction format to separate signature data (witness) from the rest of the transaction. This change increases transaction capacity, improves scalability, and fixes bugs related to transaction malleability.

Table 2: Raw transaction format (SegWit – BIP-141)

Field	Bytes	Description
Version	4	Version (little-endian)
Marker	1	(e.g., 0x00)
Flag	1	(e.g., 0x01)
TxIn Count	varint	Number of inputs
TxIn[i]:	variable	Inputs
TxOut Count	varint	Number of outputs
TxOut[j]:	variable	Outputs
Witness	variable	Witness data
LockTime	4	Lock time of block/transaction

- **marker = 0x00:** byte placed immediately after the **Version** field. It does not represent a version but functions as a *marker* signaling the start of the SegWit format and the presence of the subsequent **flag** (meaning it indicates that the transaction contains separated witness data).
- **flag = 0x01:** byte placed right after the **marker** indicating the presence of witness data, i.e., segregated signatures.
- **TxIn Count:** as in the legacy format, indicates the number of transaction inputs.
- **TxIn[i]:**
 - **Prev Hash:** hash of the previous transaction from which the output is being spent.
 - **Prev Index:** index of the output.
 - **ScriptSig Len:** length of the ScriptSig (often empty in SegWit).
 - **ScriptSig:** script providing proof of ownership of the output (often empty in SegWit).
 - **Sequence:** field used for advanced features like RBF (Replace-By-Fee).
- **TxOut Count:** as in the legacy format, indicates the number of transaction outputs.
- **TxOut[j]:**
 - **Value:** value of the output in satoshis (1 BTC = 100,000,000 satoshis).
 - **ScriptPubKey Len:** length of the ScriptPubKey.
 - **ScriptPubKey:** script specifying the conditions to spend the output (e.g., recipient's address).
- **witness:** segregated after all txs (contains digital signatures of the sender and the corresponding public keys to verify them).
- **Weight:** metric introduced by SegWit to limit the effective block size by separate accounting of witness data. It is calculated as:

$$\text{weight}(\text{tx}) = 4 \cdot \text{stripped_size}(\text{tx}) + \text{witness_size}(\text{tx})$$

Where `stripped_size` is the transaction size without witness data (equivalent to the "legacy" size) and `witness_size = total_size - stripped_size`. The block limit is expressed in "weight units" (WU):

$$\sum_{tx \in \text{block}} \text{weight}(tx) \leq 4\,000\,000 \text{ WU}$$

Furthermore, "virtual size" (`vsize`) is defined, useful for comparing legacy and SegWit transactions:

$$\text{vsize} = \left\lceil \frac{\text{weight}}{4} \right\rceil$$

Practically, SegWit counts non-witness bytes four times more than witness bytes; this incentivizes the adoption of more compact witnesses and allows packages with large witness data without excessively penalizing the `vsize` limit.

6 Resources:

- Satoshi Nakamoto's Paper (Whitepaper): <https://bitcoin.org/bitcoin.pdf>
- Bitcoin Core (repo): <https://github.com/bitcoin/bitcoin>
- Explorer / real-time blocks:
 - Mempool.space (real-time mempool and block visualization): <https://mempool.space>
 - Blockstream Explorer: <https://blockstream.info>
 - Blockchair (multi-chain search & analytics): <https://blockchair.com/bitcoin>
 - Blockchain.com Explorer: <https://www.blockchain.com/explorer>
- Useful sites for developers and documentation:
 - Bitcoin Developer Guide: <https://developer.bitcoin.org/devguide/>
 - Bitcoin Optech (best practices & guide): <https://bitcoinops.org>
 - BIP (Bitcoin Improvement Proposals) repository: <https://github.com/bitcoin/bips>
 - Bitcoin Stack Exchange (Q&A): <https://bitcoin.stackexchange.com>